

Aula 5 — IA Avançada em Testes

"IA vai substituir QA?"

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · 22/10/2026

Recap das Aulas 1–4

- Fundamentos + Playwright + SPA/PWA + Maestro

Agora chegamos onde IA reconfigura QA.

Se você dominou as 4 aulas anteriores, está preparado.

Agenda da aula (3h30)

- **90min** — IA em testes: Visual AI, geração de testes com LLM, pitfalls, self-healing (teoria + demo guiada)
- **15min** — Intervalo
- **90min** — Lab hands-on em sala: pipeline IA completo (live coding com replicação pelos alunos)
- **15min** — Wrap-up

Esta aula tem **mais densidade** que as outras. Prepare-se.

Objetivos M5

- **Aplicar** Visual AI (Applitools, Percy)
- **Construir** pipeline de geração de testes via Claude API com healing loop
- **Implementar** self-healing tests (Healenium)
- **Operar** AI agents para exploratory testing (Browser Use, Computer Use)
- **Avaliar** criticamente limites de IA em testing (oracle problem, alucinação, custo)

Por que IA reconfigura QA agora

Em 2024–2026, três avanços simultâneos:

1. **LLMs com vision multimodal** (Claude, GPT-4o) operam UI
2. **Computer Use API** (Anthropic) permite agentes navegarem app
3. **Visual AI ML-based** (Applitools) suprime falsos positivos

Cada um isolado é útil. **Combinados** mudam o que QA pode entregar em horas vs semanas.

Visual AI — paradigmas

Paradigma	Como funciona	Falso positivo
Pixel diff	Compara cada pixel	Alto (anti-aliasing, sub-pixel)
Perceptual diff (CIE Delta E, SSIM)	Modelo de percepção humana	Médio
ML-based (Applitoools)	Modelo treinado em diffs anotados	Baixo

Em 2026, **ML-based** é default em times sérios.

Applitools Eyes — exemplo Playwright

```
import { Eyes, Target } from '@applitools/eyes-playwright';

test('login screen visual', async ({ page }) => {
  const eyes = new Eyes();
  await eyes.open(page, 'MyApp', 'Login');
  await eyes.check('login-form', Target.window().fully());
  await eyes.close();
});
```

Trade-off: pode suprimir bug visual real. Auditar reports.

Geração de testes via LLM — abordagens

1. **De user story** → teste E2E
2. **De OpenAPI spec** → contract test
3. **De recording (vídeo)** → reprodução determinística
4. **De código de produção** → unit tests

Cada uma tem armadilhas distintas.

Geração via Claude API — exemplo

```
import Anthropic from '@anthropic-ai/sdk';

const client = new Anthropic();

const userStory = `
Como usuário autenticado, quero adicionar produto ao carrinho
e ver toast de "Adicionado com sucesso" após o tap.
`;

const response = await client.messages.create({
  model: 'claude-opus-4-7',
  max_tokens: 2048,
  messages: [{
    role: 'user',
    content: `
Gere teste Playwright para esta user story.
Use locators recomendados (role, text, test-id).
Use Page Object Model.

User story:
${userStory}
`
  }],
});
```

Prompt engineering — Few-shot + ReAct

Você é especialista em Playwright. Gere teste para user story.

EXEMPLO 1:

US: "Como user, quero fazer login com email/senha."

Teste:

```
test('login with valid credentials', async ({ page }) => {  
  await page.goto('/login');  
  await page.getByPlaceholder('Email').fill('user@email.com');  
  await page.getByPlaceholder('Senha').fill('secret');  
  await page.getByRole('button', { name: 'Entrar' }).click();  
  await expect(page).toHaveURL('/dashboard');  
});
```

EXEMPLO 2:

[outro exemplo similar]

US ATUAL: <user story aqui>

Teste:

Oracle problem — fundamental

Mesmo teste sintaticamente correto, executando sem erro, pode **verificar coisa errada**.

Exemplo:

```
// Teste gerado por LLM
test('checkout completes', async ({ page }) => {
  await page.click('button:has-text("Comprar")');
  await expect(page.getByText('Sucesso')).toBeVisible();
});
```

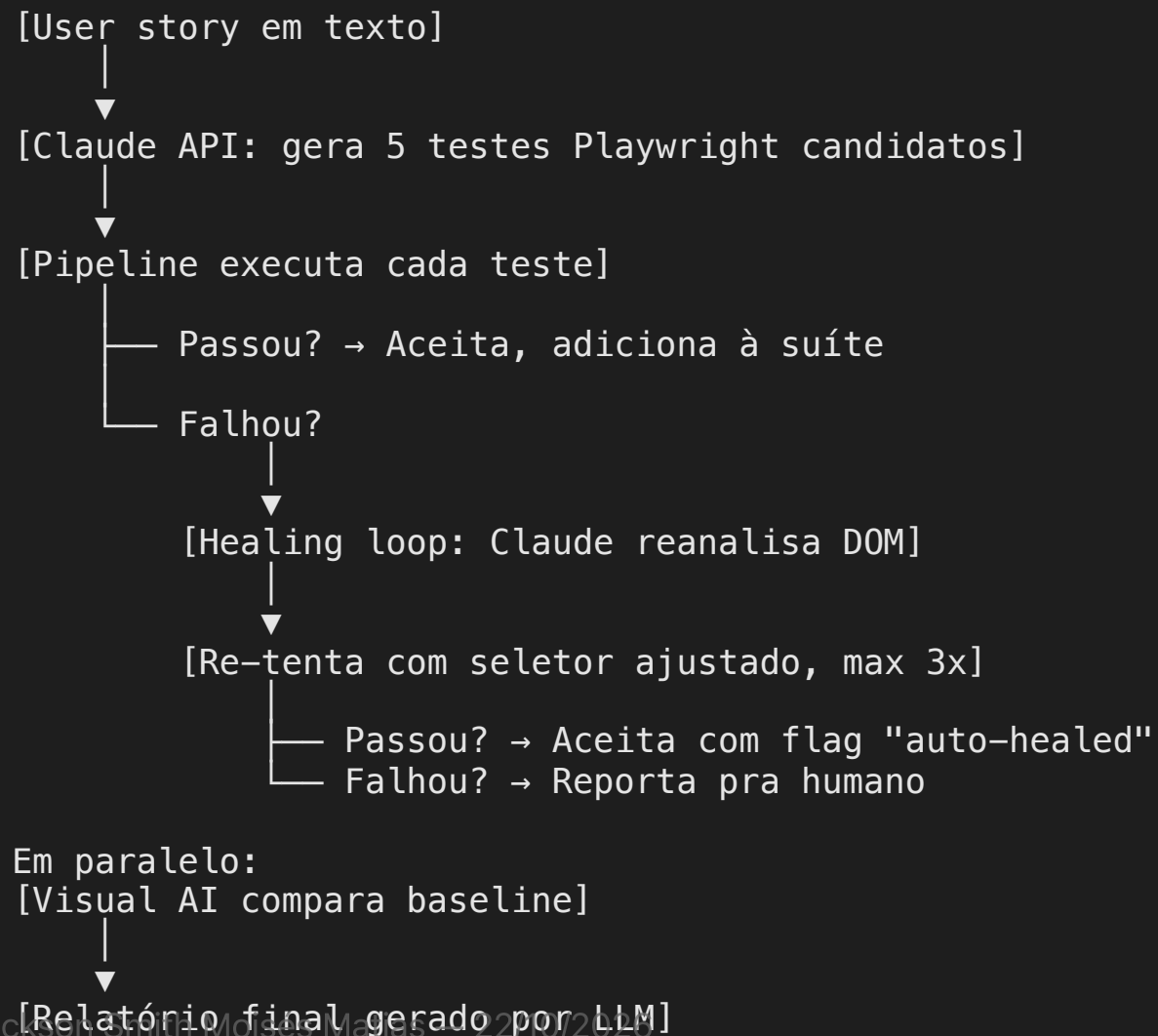
O teste "passou". **Mas e se** o app deveria mostrar valor cobrado, número do pedido, e-mail enviado? O assertion foi superficial.

Mitigando oracle problem

1. **Cross-reference com specs estruturadas** (OpenAPI, Gherkin)
2. **Revisão humana** sempre, especialmente em assertions
3. **Property-based testing** complementar — gera mil casos, espera invariantes
4. **Mutation testing** — prova que assertions detectam mudanças

| IA escreve. Humano valida.

Pipeline IA completo — desenho



Pipeline IA — implementação simplificada

```
async function generateAndHealTest(userStory: string) {  
  let attempt = 0;  
  let testCode = await generateTest(userStory); // Claude API  
  
  while (attempt < 3) {  
    const result = await runTest(testCode);  
    if (result.passed) return testCode;  
  
    // Healing  
    const dom = await captureDOM();  
    testCode = await healTest(testCode, result.error, dom); // Claude API  
    attempt++;  
  }  
  
  throw new Error('Failed to heal after 3 attempts. Human review needed.');
```

Self-Healing tests — Healenium

```
# Healenium adapter para Selenium/Playwright/Maestro
healenium:
  recovery-tries: 3
  similarity-threshold: 0.8
  log-level: WARN
```

Quando seletor quebra:

1. Healenium captura DOM
2. Busca elemento similar (heurística + ML)
3. Atualiza seletor
4. Continua execução

Risk: seletor "curado" pode apontar pra elemento errado.

Sempre auditar healing logs. Não confiar cego.

Synthetic test data via LLM

```
const fixtures = await client.messages.create({  
  model: 'claude-opus-4-7',  
  messages: [{  
    role: 'user',  
    content: `  
Gere 20 fixtures realistas de usuários brasileiros para teste:  
- nome, email, telefone (formato BR), cpf (válido)  
- inclui edge cases (nomes longos, acentos, hyphens, apóstrofes)  
- saída em JSON  
`  
  }],  
});
```

IA boa em gerar variedade realista. Cuidado com dados pessoais reais (sempre fake/sintéticos).

AI Agents — Browser Use

Open source, Python. Combina Playwright + LLM.

```
from browser_use import Agent
from langchain_anthropic import ChatAnthropic

llm = ChatAnthropic(model="claude-opus-4-7")

agent = Agent(
    task="Compre um produto qualquer no site exemplo.com. Tente cupons inválidos antes. Reporte qualquer comportamento estranho.",
    llm=llm,
)

result = await agent.run()
```

Exploratory testing automatizado. Achados não scriptados.

AI Agents — Anthropic Computer Use

API oficial Anthropic. Modelo controla desktop/browser via screenshot + ações.

```
import anthropic

client = anthropic.Anthropic()

response = client.messages.create(
    model="claude-opus-4-7",
    max_tokens=1024,
    tools=[{"type": "computer_20241022", "name": "computer"}],
    messages=[{"role": "user", "content": "Abra app exemplo, faça login, encontre bugs visuais."}]
)
```

Mais poderoso, mais caro, mais lento. Bom para exploratory profundo.

Skyvern — agente exploratory open source

```
pip install skyvern  
skyvern run-task --url https://exemplo.com --task "Faça checkout com cartão teste"
```

Casos de uso:

- Regression discovery
- Accessibility issues discovery
- Onboarding fluxos novos sem script

| Pesquise antes de adotar — projetos AI mudam rápido.

Lab principal — pipeline IA completo (90min)

Roteiro

1. Setup Anthropic Claude API key (5min)
2. Aluno fornece user story em texto (5min)
3. Implementa função que chama Claude API (15min)
4. Pipeline executa testes gerados (10min)
5. Healing loop quando falha (15min)
6. Visual AI integrado (Applitools free tier) (15min)
7. Relatório final via LLM (10min)
8. Discussão de limites (15min)

Repo starter: github.com/jacksonsmith/puc-iec-qualidade-web-mobile/tree/main/starters/modulo-05

Pitfalls comuns

- ✗ **Aceitar testes IA sem revisar** — oracle problem destrói confiança
- ✗ **Healing sem auditoria** — bugs reais ficam mascarados
- ✗ **Custo descontrolado** — `tracesSampleRate: 1.0` em prod = \$\$\$
- ✗ **Visual AI tolerância demais** — bug visual real escapa
- ✗ **Agente em loop infinito** — sempre limite tentativas

Trade-offs IA em QA — honesto

Quando IA brilha:

- Transformação (story → teste, imperativo → POM)
- Exploratory testing autônomo
- Análise interpretativa de Lighthouse reports
- Synthetic data realista

Quando IA falha:

- Decisão arquitetural de teste
- Oracle problem (saber o que assertar)
- Edge cases sutis específicos do domínio
- Casos onde "funciona" não é o critério

Atividade — Lab IA Test Gen (15 pts)

Entrega: ver Canvas.

Entregar repo com:

1. **Pipeline funcional** (story → test → execução) sem intervenção manual — 6pts
2. **Healing loop** implementado (re-prompt em failure) — 4pts
3. **Relatório crítico** (3pgs) identificando limites observados — 5pts

Pontos do relatório:

- Tipos de teste em que IA acertou
- Tipos em que falhou
- Custo em tokens / latência observada
- Comparação com testes escritos à mão

Leituras obrigatórias

- **Anthropic** — *Prompt Engineering Guide + Computer Use docs*
- **Applitools** — *Visual AI Whitepaper* (2024)
- **Yang, J. et al.** (2023) — *LLMs for Software Testing: A Survey*. arXiv:2307.07221
- **Wang, J. et al.** (2024) — *Software Testing with LLMs*. IEEE TSE
- **Tufano et al.** (2022) — *Methods2Test*. MSR
- **Schäfer et al.** (2024) — *Empirical Eval of LLMs for Unit Test Gen*. IEEE TSE

Próximo módulo (M6)

Cross-Browser, CI/CD & Projeto Final

- Cross-browser strategies
- Mobile farm strategies
- Test Impact Analysis
- Quality gates (SonarQube, Stryker)
- DORA metrics aplicadas a QA
- **Projeto Final**

Pré-requisito: Lab IA entregue + começar projeto final.

Próximo módulo!

 jackson.96@gmail.com

 [linkedin.com/in/3jacksonsmith](https://www.linkedin.com/in/3jacksonsmith)

 github.com/jacksonsmith